Clearway in brief

Airport Investment Intelligence Agent, Deloitte Digital Forward Deployed Engineer exercise

Clearway is an AI agent that helps analysts at an airport-modernization investor screen US airports for terminal expansion. Claude understands the question, picks an analysis and explains the result; deterministic TypeScript computes every number from public aviation data, and each answer shows its period, sources, assumptions and confidence.

Live demo: clearway-jade.vercel.app
Code: github.com/eliran0220/clearway
Methodology in detail: docs/DESIGN.md

What it answers

Question type	How Clearway answers
Expansion candidates in a region	A 0 to 100 screening score: demand growth 35, seat occupancy 30, delay pressure 20, supply gap (passengers outgrowing seats) 15, each on a fixed range so a score does not depend on which airports are compared
Congestion between airports	Share of arrivals 15+ minutes late, cancellations and the air-traffic-system share of delays over the same 12 months, plus documented caps and live FAA status
Long-haul share	Departures of 4,000 km or more as a share of all departures, with the threshold and cargo inclusion adjustable
Unmet demand	No public data counts travelers who could not fly, so four measured signals instead: planes fuller than the national average, passengers outgrowing seats, strained operations, documented capacity limits
Follow-ups and out-of-scope	"Less weight on delays" or "now 5,000 km" re-run the tool; ROI or cost questions are declined with the data they would need

How we worked

1. **Research and plan.** We surveyed free public aviation sources and cross-checked the findings with a second model (ChatGPT) before choosing any.
2. **Proof of concept.** A separate POC called the real endpoints (BTS traffic API, BTS file exports, FAA status, NOAA weather) to confirm fields, coverage and freshness. It showed no free API has route distances or delay causes, so those two sources are official file imports.
3. **Implementation plan.** The POC results went into one plan covering architecture, scoring, data windows, deployment and tests, with each decision recorded.
4. **Build and verify.** Built in stages; 98 unit tests; a model evaluation that fails any answer quoting a number no tool returned (8 of 8 pass); a browser pass at desktop and phone width; a sanity check against the live URLs; architecture, implementation and code reviews.

Stack

Layer	Choice	Why	Instead of
Language	TypeScript, npm workspaces	One language and shared types for agent, scoring and UI	Python: stronger data tools, but a second language and deploy
Agent	Claude Sonnet via the Vercel AI SDK	Typed, validated tools; a bounded tool loop; streaming to the UI	LangChain: more than one tool loop needs. RAG: answers are calculations, not document search
Scoring	Plain TypeScript package, unit-tested	Deterministic and explainable component by component	Ranking by the model: not reproducible
API	Hono on AWS Lambda, streaming URL	Fast start, streams answers, pay per request	A server on EC2: always-on cost
Infrastructure	SST	API, daily job and secrets in one file	CDK or Terraform: more code for the same
Database	Postgres on Neon, Drizzle	Relational monthly data, SQL totals, scales to zero	Aurora: slower to wake. MongoDB: data is tabular
Data	BTS API (daily job), BTS files, FAA and NOAA live	Public, official, citable	Paid feeds such as OAG: richer, not public
Web and voice	Next.js on Vercel; browser speech	Preview per push; no audio leaves the browser	Server transcription: cost for a bonus

Advantages and limits

Advantages. Every number traces to a source, period and formula. Answers state assumptions, confidence and what the data cannot show. Running cost is close to the model calls alone. Small surface: one language, one repo, CI on every push.

Limits. A screening tool, not an ROI model: public data has no construction costs or airport revenues. Route and delay files are refreshed by hand; only traffic refreshes daily. Score weights are reasoned, not calibrated against past projects. The rate limit is per server instance; production would add a shared limit and sign-in.

What we'd build next

- **Watchlist alerts.** Analysts follow airports or regions; when BTS publishes a new month, the existing daily job emails or posts to Slack only what changed ("Portland moved from 3rd to 2nd"), so nobody re-asks the same questions each month.
- **One-click committee memo.** Export any answer as a PDF or Word memo with its cards, sources and assumptions, so numbers reach the investment committee with their evidence attached.
- **FAA forecasts in the score.** Add the FAA Terminal Area Forecast so the ranking weighs projected demand over the next decade, not only the last two years: the closest public data gets to "most profitable".